

Plan First, Diffuse Later: Extrinsic Graph Guidance for Long-Horizon Diffusion Planning

Yaniv Hassidof*
Technion

Adir Morgan
Technion

Yilun Du
Harvard

Kiril Solovey
Technion

Abstract

Compositional diffusion models offer a promising route to long-horizon planning by denoising multiple overlapping sub-trajectories while ensuring that together they constitute a global solution. However, enforcing local behavior over long chains is often insufficient for a coherent global structure to emerge. Recent works tackle this limitation through *intrinsic* search, which explores multiple paths *during* the denoising process. While intrinsic search improves global coherence, it comes at the cost of repeated evaluations of an already compute-heavy model. In this work, we argue that *extrinsic* search, performed *outside* the denoising process, offers a more effective mode of exploration for long-horizon planning while naturally enabling the use of classical algorithms to solve unseen combinatorial tasks at test time. Our eXtrinsic search-guided Diffuser (XDiffuser) first computes a plan over a state-space graph—serving as a lightweight local connectivity oracle for the diffusion model. The plan is then used to guide denoising for a *single* trajectory, effectively offloading the burden of exploration. XDiffuser outperforms diffusion-based baselines on long-horizon tasks, with particularly large gains in the low-quality data regime and on unseen tasks beyond goal-reaching, including multi-agent coordination and TSP-style reasoning. Website: <https://yanivhass.github.io/XDiffuser-site/>

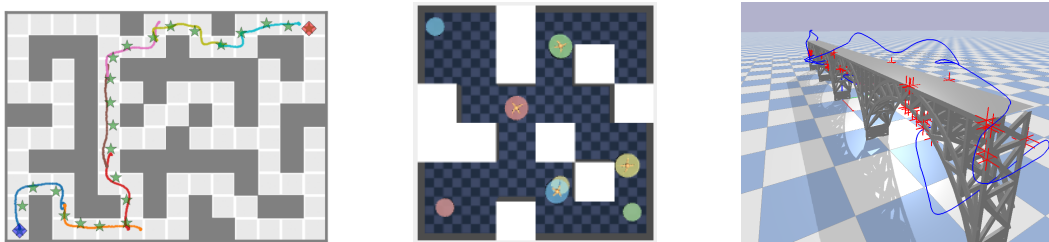


Figure 1: By leveraging task-specific graph-search mechanisms, XDiffuser enables a pretrained goal-reaching compositional diffuser to solve complex unseen tasks. **Left:** Waypoints endow local segments with a coherent global structure, strengthening long-horizon goal reaching. The agent is marked by $\blacklozenge$, waypoints by $\star$, and the goal by $\blacklozenge$. **Middle:** Multi-agent goal-reaching AntMaze. **Right:** XDiffuser performs an inspection planning task when paired with an existing graph-based solver. Points of interest for inspection are marked in red, XDiffuser’s trajectory shown in blue.

1 Introduction

Learning-based planning is pushing the frontiers of robotic decision-making, extracting effective behavior from static datasets collected by short-horizon, often suboptimal policies. However, scaling

*Corresponding author. Email: yaniv_hass@campus.technion.ac.il

these methods to long-horizon tasks such as robotic assembly [Jiang et al., 2022], navigation [Nahavandi et al., 2025] and infrastructure inspection [Lattanzi and Miller, 2017] remains a central challenge [Park et al., 2025a], as both learned dynamics models [Moerland et al., 2023] and regressed value-based policies [Wang et al., 2024] suffer from compounding approximation errors.

Diffusion-based planners [Janner et al., 2022] offer an appealing alternative by learning to directly sample a coherent trajectory from the data distribution, thereby casting planning as inference [Botvinick and Toussaint, 2012]. Yet, planning-as-inference faces its own limitations in the long-horizon regime. Real-world demonstrations required for training are often short, local, and incomplete due to the monetary, time, and safety constraints of data collection, thus placing long-horizon problems inherently outside the data distribution. To address this limitation, recent approaches leverage *compositionality* [Mishra et al., 2023, Luo et al., 2025] to assemble short-horizon trajectory segments that are mutually compatible into a coherent global long-horizon solution. However, as the horizon grows, compatibility must be maintained over an increasingly long chain of segments, each attempting to maximize its own local likelihood, making the inference process increasingly brittle: the model may generate locally plausible segments that are mutually incompatible, leading to failures in global consistency, task completion, or constraint satisfaction.

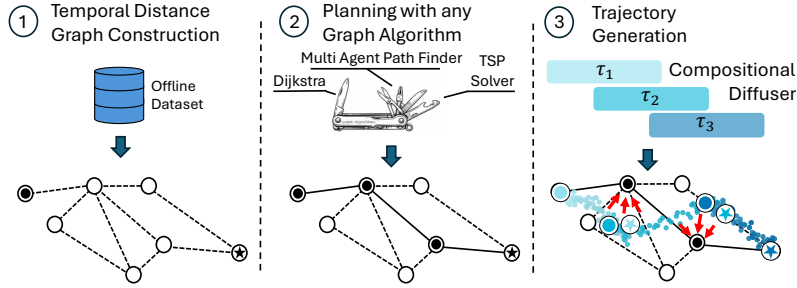


Figure 2: XDiffuser decomposes planning into *extrinsic* search followed by guided *intrinsic* generation. (1) At training time, a temporal distance representation is used to construct a connectivity graph over sampled dataset states. (2) A task-appropriate graph search is executed, producing a sequence of waypoints representing the graph solution. (3) A pretrained CompDiffuser denoises a smooth trajectory, guided by the waypoints set.

We argue that this limitation is not merely a consequence of approximation error during inference, but a fundamental structural limitation of pure planning-as-inference approaches: when the target long-horizon distribution is never observed, local consistency alone is insufficient to guarantee globally-coherent behavior. Our key insight is that these requirements are naturally handled by *extrinsic* search (i.e., *outside* of denoising), while diffusion is best suited to synthesizing smooth, dynamically plausible trajectories between compatible states. Based on this view, we introduce the eXtrinsic search-guided Diffuser (XDiffuser), which is visualized in Figure 2. Concretely, at training time, XDiffuser builds a graph over real trajectory data to produce a sparse, globally consistent scaffold. At test time, XDiffuser *first* searches this graph scaffold to produce a high-level plan that is *then* used for guiding a pretrained compositional diffusion planner to generate continuous trajectories that are smooth, likely under the data, and executable. In contrast to prior approaches that perform search *inside* the denoising process [Zhang et al., 2025, Mishra et al., 2026, Yoon et al., 2025], XDiffuser avoids repeatedly querying the model during iterative denoising, making planning substantially more efficient; we discuss this distinction in more detail in the related work.

This decomposition between search and denoising yields a simple and modular framework for long-horizon planning. As the search layer operates over a structured graph it naturally supports adaptive horizon selection: the diffusion horizon is inferred from the temporal duration of the selected route, rather than fixed in advance by a heuristic rollout length as in prior methods. Furthermore, since the high-level objective lives in the search procedure, different planning algorithms can be incorporated at test time, enabling extensions beyond standard goal-reaching to settings such as multi-agent coordination or inspection planning (Figure 1).

Our experimental results (Section 5) highlight XDiffuser’s test-time scalability. On the OGBench suite [Park et al., 2025b], XDiffuser exhibits strong long-horizon planning, especially after training on low-quality data, by achieving 98.5% success rate on AntMaze Large Explore—over 70% higher than its base diffusion planner. We then demonstrate strong generalization across task structure while

reusing the same pretrained diffuser in two tasks: (i) a multi-agent setting, where the single-agent diffuser is paired with priority-based graph planner to enable coordinated behavior, and (ii) inspection planning, leveraging a TSP-style graph planner.

2 Related Work

Planning with Graphs. Planning over explicit graphs is one of the oldest, most widespread paradigms for long-horizon decision making. In robot motion planning, search is often carried out on a graph [Cohen et al., 2011, Kavraki et al., 1996, Panasoff and Solovey, 2025], capturing the problem’s state-space connectivity, to generate efficient collision-free paths. Moreover, state-space graphs are leveraged in obtaining complex behaviors beyond standard goal reaching, such as multi-robot coordination [Wang et al., 2025] or inspection planning [Morgan et al., 2026].

Graph search has been widely combined with reinforcement learning (RL) for long-horizon planning. Prior work plans over graphs of observations or learned landmarks, using image-space graphs [Savinov et al., 2018, Liu et al., 2020], value-guided subgoal search [Eysenbach et al., 2019], or latent landmarks with multi-step edges [Zhang et al., 2021]. More recently, Graph-Assisted Stitching (GAS) [Baek et al., 2025] performs search in a latent temporal-distance space. These approaches decompose long-horizon problems into locally feasible steps, but rely on strong value-based policies to track graph waypoints. However, this abstraction is inherently limited. Graph edges provide only local, approximate feasibility—indicating that transitions are possible or observed—without guaranteeing that sequences of edges form likely trajectories under the data distribution. As a result, individually valid transitions may compose into globally unnatural behaviors [Zhang et al., 2021], and shortest-path objectives can favor brittle plans that cut corners or traverse rarely co-occurring states.

To address these shortcomings, our graph-guided diffusion approach takes inspiration from robot autonomy pipelines and hierarchical motion planning approaches wherein a high-level graph plan is used to invoke a mid-level trajectory optimization approach to produce long-horizon dynamically feasible trajectories [Betz et al., 2023, Wahba et al., 2024, Huang et al., 2025]. However, in our setting, rather than refining trajectories only via gradients of hand-designed costs and assumed-known obstacles, the diffusion model leverages a learned score function to synthesize trajectories that are smooth, dynamically feasible, and consistent with the data distribution.

Planning with Diffusion. Diffusion models [Ho et al., 2020] have emerged as expressive trajectory priors and planners. Diffuser [Janner et al., 2022] formulates planning as iterative denoising over entire trajectories, achieving strong performance while enabling controllability through guidance [Dhariwal and Nichol, 2021], yet remaining limited to the horizon lengths observed during training. Generative Skill Chaining (GSC) [Mishra et al., 2023] and CompDiffuser [Luo et al., 2025] aim to synthesize long-horizon behavior from short demonstrations by stitching overlapping denoised segments. In practice, however, maintaining global structure over long horizons remains difficult as denoising of each segment only follows a local consistency objective. A common failure case of such methods is sampling of neighboring segments from incompatible modes of the trajectory distribution, leading to invalid states at segment overlaps due to mode averaging [Mishra et al., 2026].

A recent line of work attempts to strengthen compositionality by injecting search directly into diffusion inference to guide long-horizon stitching [Zhang et al., 2025, Yoon et al., 2025, Mishra et al., 2026], iteratively calling the diffusion denoiser to search over possible denoising steps. These methods show that additional test-time compute can substantially improve diffusion planning, but they also reveal a key bottleneck: when search is placed *inside* denoising, the computational cost scales with both search complexity and diffusion depth. In the worst case, a search procedure with branching factor b , depth H , and K denoising steps per proposal requires $O(Kb^H)$ denoiser calls [Russell and Norvig, 2009]. Compositional Diffusion with Guided Search (CDGS) [Mishra et al., 2026] reduces this cost by maintaining a fixed-sized population of candidate plans that is iteratively resampled and pruned, but by doing so, it sacrifices explicit backtrack-capable search structure.

Compositional Monte-Carlo Tree Diffusion (C-MCTD) [Yoon et al., 2025] performs tree search via an *online composer*, where nodes represent partial trajectories and edges correspond to candidate plan extensions. To reduce the burden of test-time denoising, C-MCTD introduces a *preplan composer*: a graph constructed during training, in which edges between dataset states are generated by querying the online composer. At inference, start and goal are connected to this graph, and shortest-path search returns a sequence of edges that are stitched into the final trajectory. However, constructing this graph

requires running the full search procedure between many state pairs, leading to a worst-case cost of $O(V^2 K b^H)$ denoiser calls for V vertices. Moreover, edges only approximately match endpoint positions (within ϵ) and ignore other state variables (e.g., velocity, orientation), which can introduce inconsistent transitions. In contrast, XDiffuser generates a single coherent and dynamically feasible trajectory via waypoint-guided diffusion. This distinction enables us to perform large-scale search, as we show in multi-agent path finding and inspection planning tasks, while successfully handling complex dynamics.

In different settings, several recent works introduce a high-level discrete scaffold precisely because pure trajectory denoising struggles with long-horizon discrete decision making: DiTree [Hassidof et al., 2025] combines a diffusion policy with state-space tree expansion for kinodynamic motion planning. DGD [Liang et al., 2026] uses discrete multi-agent paths to guide continuous diffusion in multi-robot planning, DiMSam [Fang et al., 2024] uses task-and-motion planning to compose learned diffusion samplers, and Hybrid Diffusion [Høeg et al., 2026] jointly models symbolic and continuous plans. In contrast to those works, our XDiffuser performs search outside of denoising using a lightweight and general-purpose planning scaffold.

3 Problem Formulation

We consider the problem of offline long-horizon motion planning for a robot operating in a continuous state space. We are given a fixed dataset $\mathcal{D}$ of previously collected trajectories, where no further interaction with the environment is allowed prior to deployment. Each trajectory is a sequence $\tau = (s_1, a_1, \dots, s_T, a_T)$, where $s_t \in \mathcal{S}$ denotes the robot’s state (e.g., position, velocity, orientation) and $a_t \in \mathcal{A}$ denotes the control input. In practice, our method requires only access to states, and actions are used to train an inverse dynamics model or derived by a controller, in accordance with existing methods [Ajay et al., 2023, Luo et al., 2025]. Importantly, the dataset consists of short-horizon behaviors collected under one or more behavior policies and does not necessarily contain trajectories that solve the long-horizon tasks of interest. Accordingly, models are trained on local windows extracted from $\mathcal{D}$, each of length at most H_{train} , and thus only capture short-term, local behavior rather than complete task solutions.

At test time, the robot is given an initial state s_{start} and a task specification, such as a goal state, cost function, or additional constraints. The objective is to generate a feasible trajectory $(s_1, \dots, s_{H_{\text{test}}})$ that satisfies the task, where typically $H_{\text{test}} \gg H_{\text{train}}$. The central challenge is therefore to compose locally valid behaviors into a globally consistent, long-horizon plan.

4 Method

We describe our XDiffuser approach for long-horizon planning from offline data. The key idea is to separate *global coordination* from *local trajectory generation*. Rather than relying solely on overlap consistency to propagate information across a long chain of locally generated segments, XDiffuser leverages a connectivity graph to first search for a coarse, globally consistent sequence of temporal waypoints. XDiffuser then uses this waypoint scaffold as a soft energy term defined over the full trajectory to guide compositional diffusion toward a coherent global solution. Overall, each segment is generated under three complementary constraints: local feasibility from the pretrained diffusion prior, consistency with adjacent segments through learned overlap coupling, and global coordination through the waypoint scaffold. We now detail each component of our method.

4.1 Connectivity Graph Construction

We now describe our construction of an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ over the offline dataset to capture coarse, task-agnostic connectivity between states, as illustrated in Algo 1. We first uniformly sample N states from the dataset $\mathcal{D}$ to form the vertex set $\mathcal{V} = \{v_1, \dots, v_N\} \subset \mathcal{S}$. To determine connectivity, we rely on a learned temporal-distance representation (TDR) $f_\psi : \mathcal{S} \rightarrow \mathbb{R}^d$ as formulated by [Park et al., 2024, Baek et al., 2025]. In this representation, Euclidean distance reflects temporal proximity between states. Importantly, TDR is used only to define distances and neighborhood structure, whereas the graph vertices lie in the original state space. We define the edge cost as $c(v_i, v_j) = \|f_\psi(v_i) - f_\psi(v_j)\|_2$. Each vertex v_i is connected to its k nearest neighbors under this cost, subject to a connectivity threshold $\|f_\psi(v_i) - f_\psi(v_j)\|_2 \leq \alpha$, which we adopt from [Baek et al.,

2025]. At test time, task-dependent states (e.g., start, goal) are added to the graph using the kNN procedure.

The graph $\mathcal{G}$ encodes only coarse reachability, i.e., which states can plausibly connect, while deferring the generation of smooth and dynamically consistent trajectories to the downstream diffusion model. As a result, the graph is scalable, reusable across tasks, and independent of the final planning objective. While more sophisticated graph construction methods could be incorporated [Zhang et al., 2021, Baek et al., 2025], we find this minimal design sufficient for guiding a long-horizon diffusion planner.

Algorithm 1 Connectivity Graph Construction

- 1: Sample N states $\mathcal{V} \subset \mathcal{D}$
 - 2: Compute TDR $z_i = f_\psi(v_i), \forall v_i \in \mathcal{V}$
 - 3: $\mathcal{E} \leftarrow \emptyset$
 - 4: **for** each $v_i \in \mathcal{V}$ **do**
 - 5: $\mathcal{N}_i \leftarrow \text{kNN of } z_i$
 - 6: **for** each $z_j \in \mathcal{N}_i$ **do**
 - 7: **if** $\|z_i - z_j\|_2 \leq \alpha$ **insert** (v_i, v_j) **to** $\mathcal{E}$
-

4.2 Planning via Off-the-Shelf Graph Algorithms

Next, the graph $\mathcal{G}$ is used to produce waypoints for downstream diffusion guidance. At test time, we are given a planning problem specified by an initial state s_{start} and a task objective, such as a goal state or constraint. We invoke a graph search algorithm to find a plan that satisfies those constraints. For example, in a goal-reaching task, a shortest path $\tau_{\mathcal{G}} := (v_0 = s_{\text{start}}, v_1, \dots, v_R = s_{\text{goal}})$ over $\mathcal{G}$ can be obtained, as illustrated in Fig 4. Since graph edge costs approximate temporal distance, the path induces a nominal timing along the route through the cumulative costs $t_r = \sum_{i=0}^{r-1} c(v_i, v_{i+1})$. However, a dense sequence of waypoints is often too restrictive for downstream diffusion of a smooth trajectory. Thus, we downsample this path $\tau_{\mathcal{G}}$ at a fixed temporal interval Δt to obtain the final sequence of temporal waypoints $\mathcal{W} = \{(w_m, \hat{t}_m)\}_{m=0}^M$, where w_m is the selected graph state and $\hat{t}_m$ is its nominal time along the route. The interval Δt determines the density of the waypoint scaffold: smaller values produce tighter control over the diffusion process, at the cost of possibly over-constraining the diffusion model to produce infeasible trajectories. An important observation is that graph shortest paths prioritize global optimality at the cost of possibly inducing dynamically infeasible nominal timing. To better align the plan with downstream diffusion, we temporally dilate our waypoints by a constant factor so that the number of generated chunks roughly matches the number of chunks reported by Luo et al. [2025].

4.3 Waypoint-Guided Compositional Diffusion

We now explain how the graph solution, captured by the temporal waypoints $\mathcal{W}$, is used as a coarse scaffold for long-horizon trajectory generation. Let $\tau = (s_1, \dots, s_H)$ denote the full trajectory to be computed using the diffusion model. Following Luo et al. [2025], τ can be decomposed into K segments $\tau_1, \dots, \tau_K$ with $\mathcal{O}$ overlapping states between every subsequent pair, represented by the approximated distribution

$$p_\theta(\tau \mid q_s, q_g) \propto p_\theta(\tau_1 \mid q_s, \tau_2) p_\theta(\tau_K \mid \tau_{K-1}, q_g) \prod_{k=2}^{K-1} p_\theta(\tau_k \mid \tau_{k-1}, \tau_{k+1}),$$

where every factor p_θ is generated by the same short-horizon diffusion model with weights θ , enabling long-horizon generation by enforcing local consistencies while neglecting non-local dependencies [Yedidia et al., 2005].

To inject the missing global structure, we leverage the waypoint sequence $\mathcal{W}$ via gradient-based diffusion guidance [Carvalho et al., 2023, Song et al., 2023]. We begin by initializing the noisy trajectory prior to denoising by treating the nominal time of the final waypoint w_M as the expected goal-reaching time. Each waypoint $w_m \in \mathcal{W}$ is then associated with a temporal region of the trajectory centered at its nominal time $\hat{t}_m$. Rather than enforcing hard interpolation through waypoints, we use them as a soft scaffold, allowing the diffusion process to produce locally plausible trajectories.

To this end, we define a triangular guidance window around each waypoint:

$$\lambda_m(t) = \max\left(0, 1 - \frac{|t - \hat{t}_m|}{r}\right),$$

where $r > 0$ is the window radius. This weight is maximal at $t = \hat{t}_m$ and decays linearly to zero with temporal distance. In order to conform with the existing overlap guidance term, for all our experiments we set the window size to match the overlap length $\mathcal{O}$. The resulting waypoint guidance energy for a trajectory τ is

$$E_{\mathcal{W}}(\tau) = \sum_{m=0}^M \sum_{t=1}^H \lambda_m(t) \|s_t - w_m\|_2^2.$$

Which induces the guided distribution

$$p_{\theta}(\tau \mid s_{\text{start}}, s_{\text{goal}}, \mathcal{W}) \propto p_{\theta}(\tau \mid s_{\text{start}}, s_{\text{goal}}) \exp(-E_{\mathcal{W}}(\tau)).$$

At each denoising step, we augment the base denoising score by the gradient of the waypoint cost:

$$\nabla_{\tau} \log p_{\theta}(\tau \mid q_s, q_g, \mathcal{W}) \approx \nabla_{\tau} \log p_{\theta}(\tau \mid q_s, q_g) - \nabla_{\tau} (E_{\mathcal{W}}(\tau)).$$

This form preserves the original compositional model while biasing denoising toward trajectories close to the graph solution.

5 Experiments

We evaluate the performance of XDiffuser to demonstrate that high-level extrinsic graph guidance improves long-horizon goal reaching. Moreover, we show that the same graph scaffold can be reused for unseen planning tasks (i.e., MAPF and inspection planning) by changing the high-level graph objective at test time. Ablation studies are discussed in Appendix A.1. Our goal-reaching and MAPF experiments are conducted in the OGBench [Park et al., 2025b] suite. For the bridge inspection experiment, we use a separate PyBullet simulation environment [Coumans and Bai, 2016]. All experiments were conducted on a single RTX 3090 GPU.

5.1 Does extrinsic graph guidance improve long-horizon goal reaching?

We evaluate generation of long goal-reaching trajectories when the base diffusion model is trained on short demonstrations alone, illustrated in Fig. 1. Moreover, we evaluate XDiffuser’s performance based on the *Explore* dataset, comprised of random-walk style demonstrations.

Baselines. We compare XDiffuser against the alternatives discussed in Section 2. **CD** [Luo et al., 2025] represents pure compositional diffusion, and shares the same underlying diffusion model weights as XDiffuser, but relies entirely on local stitching without explicit search. **CDGS** [Mishra et al., 2026] augments CD with a population-based search over candidate trajectories during denoising, reinforced by multiple resampling iterations. **C-MCTD** [Yoon et al., 2025] places search directly inside denoising via tree search, spending inference-time compute on branching over partially denoised rollouts. Finally, **GAS** [Baek et al., 2025] is a strong graph-based baseline that uses a value-based policy to follow the shortest path over a pruned and clustered temporal distance graph. This makes GAS a useful comparison point for isolating the benefit of combining graph structure with a generative trajectory model.

Evaluation Setup. We report *execution success rate*: a rollout is successful if the robot reaches the goal position in the maze within the episode time limit. For each environment, we evaluate on five different start–goal tasks, each evaluated with 20 episodes, and the full evaluation is repeated over five random seeds. The mean and std are computed across seeds. All methods are evaluated on the same OGBench goal-reaching protocol, where the policy must execute the generated or planned trajectory in the environment rather than merely produce a geometrically-valid plan.

Results. Table 1 shows XDiffuser substantially improves long-horizon goal reaching over pure CD. The improvement is most pronounced in settings where local stitching is insufficient: the *Giant* maze and the *Explore* datasets. On *AntMaze Explore Large*, for example, CD succeeds only 27% of the time, whereas XDiffuser reaches 98.5% success. This gap suggests that when demonstrations are suboptimal and do not directly provide clean expert-like paths, relying only on probabilistic inference is insufficient. In contrast, a graph allows XDiffuser to identify promising waypoints before invoking the diffusion model. Compared C-MCTD and CDGS, XDiffuser allocates inference-time search to a compact high-level graph rather than branching over partially denoised trajectories, which becomes increasingly important as the required horizon grows. Branching directly within denoising

Environment	CD	C-MCTD	CDGS	Ours	GAS
PointMaze (Stitch Giant)	68 $\pm$ 3	100 $\pm$ 0	82 $\pm$ 4	100 $\pm$ 0	–
AntMaze (Stitch Giant)	65 $\pm$ 3	75 $\pm$ 18	84 $\pm$ 3	90.0 $\pm$ 2.2	88.3 $\pm$ 3.6
AntMaze (Stitch Large)	86 $\pm$ 2	94 $\pm$ 9	–	90.6 $\pm$ 2.2	96.3 $\pm$ 0.9
AntMaze (Stitch Medium)	96 $\pm$ 2	98 $\pm$ 6	–	93.0 $\pm$ 3.0	98.1 $\pm$ 1.2
AntMaze (Explore Large)	27 $\pm$ 1	–	–	98.5 $\pm$ 0.5	94.2 $\pm$ 3.0
AntMaze (Explore Medium)	81 $\pm$ 2	–	–	99.3 $\pm$ 0.5	94.2 $\pm$ 3.0

Table 1: Success rate comparison (mean $\pm$ std) across OGBench environments. Results for other methods are taken from their respective papers. Results not reported are marked as blanks (–).

can be expensive and unstable, since each partial rollout must remain dynamically plausible while also making progress toward a distant goal. Moreover, denoising is *time-consuming*—CDGS took on average $10\times$ longer (on our hardware) to generate, in accordance with the multiple resampling steps it performs. Since C-MCTD does not have a publicly available implementation, we could not measure its runtime on our hardware. However, on their much stronger hardware (8 NVIDIA RTX 4090 GPUs) they report runtimes which are $5\times$ longer than XDiffuser for Pointmaze *Giant* on our setup. Additionally, as evident in their reporting, runtime scales exponentially as planning horizon grows due to their branching factor, indicating an even larger gap on more complex tasks.

The results also reveal where graph guidance is less critical. On the shorter and easier *Stitch Medium* and *Stitch Large* tasks, the base CD model is already strong, reaching 96% and 86% success respectively. In these settings, waypoint guidance provides little improvement and can even slightly reduce performance when waypoints over-constrain an already reliable denoiser. This suggests that XDiffuser is most beneficial when the task requires genuine long-horizon planning.

GAS provides an important comparison since it also uses a graph planner, but without a diffusion planner, and achieves the best results on AntMaze *Stitch Medium* and *Stitch Large*. However, XDiffuser is stronger on the most challenging *Explore* settings and on AntMaze *Stitch Giant*, highlighting the robustness of XDiffuser as problem horizon grows, as well as the advantage of fine-grained trajectory guidance which we evaluate in the following section.

5.2 Does extrinsic graph search generalize to unseen tasks?

We test the modularity of the graph layer, by fixing the learned diffusion model and graph structure, but changing the high-level planning objective at test time. We focus on MAPF and inspection planning as two representative unseen tasks; both require combinatorial reasoning not learned by the model, but which could be accommodated by graph algorithms.

5.2.1 Can search alone unlock zero-shot reasoning with diffusion?

We consider multi-agent path finding (MAPF) on AntMaze *Stitch Medium* with $n \in \{2, 3, 4\}$ ant robots, shown in Fig. 1. Each agent i is assigned a start-goal pair $(s_{\text{start}}^i, s_{\text{goal}}^i)$, and the planner must output a collection of state trajectories $\{\tau^i\}_{i=1}^n$, where $\tau^i = (s_1^i, \dots, s_T^i)$. A joint plan is successful if every agent reaches its goal and all pairwise collision constraints are satisfied: for every pair $i \neq j$ and every timestep t , we require $\|pos(s_t^i) - pos(s_t^j)\|_2 \geq \delta$, with collision threshold δ , where pos is the position of the robot. This setting is intentionally out of distribution for a diffusion model trained only on single-agent data and never observes interaction constraints during training. We evaluate 20 episodes, over 3 random seeds each, where in each episode each agent executes one of the five queries from the single agent AntMaze environment. We report success rate where a success is defined by all agents reaching their destination with no collisions.

Our method. We instantiate XDiffuser with a multi-agent priority-planner [Erdmann and Lozano-Perez, 1987] as **PP-XDiffuser**, in which each agent first solves a prioritized planning problem on its respective graph. Planning is still based on a shortest path, but each agent treats higher priority agents as dynamic obstacles on the graph. The resulting waypoint sequences are used to guide diffusion generation of each agent, as in the goal-reaching setting.

Baselines. We compare 4 other methods of utilizing the same pretrained single-agent model at test time. **Naive CD** and **Naive CDGS** plan independently for each agent using CD or CDGS,

respectively, and ignore collisions. **Guided CD** evaluates the effect of diffusion guidance without search: following Shaoul et al. [2025], it applies prioritized repulsion guidance during denoising, so that each agent is repelled only from the trajectories of higher-priority agents. This makes it the guidance-only analogue of prioritized MAPF, but without any explicit search over a separate graph structure. **Prioritized CDGS** augments CDGS’s population-based search by penalizing trajectories in collision with higher priority agents. This setup isolates our main question: can explicit search induce zero-shot multi-agent coordination from a purely single-agent diffusion prior? All methods share the same model weights, and denoise three segments, following Luo et al. [2025].

Environment	#Agents	Naive CD	Naive CDGS	Guided CD	Prioritized CDGS	PP-XDiffuser
AntMaze Stitch Medium	2	38.33 ± 6.24	48.33 ± 2.36	40.00 ± 4.08	65.00 ± 8.16	88.33 ± 6.24
	3	3.33 ± 4.71	10.00 ± 4.08	15.00 ± 10.80	30.00 ± 4.08	73.33 ± 2.36
	4	0.00 ± 0.00	1.67 ± 2.36	1.67 ± 2.36	13.33 ± 6.24	58.33 ± 13

Table 2: Performance comparison on AntMaze Stitch Medium with varying numbers of agents.

Results. Table 2 demonstrates that search is a crucial component for handling multi-agent coordination zero-shot. As the number of agents grows, naive single-agent plans fail frequently, and Guided CD’s purely-local repulsive guidance is insufficient to prevent deadlocks and collisions. In contrast, only the methods that embed search, Prioritized CDGS and PP-XDiffuser, can coordinate 4 agents. However, XDiffuser’s extrinsic search offers a much more effective alternative by first rapidly forming a coarse global plan which satisfies the new task constraints, and only then initiating the denoising process. As a result, for 4 agents XDiffuser achieves 58% success rate compared with CDGS’s 13%, as well as being more efficient—with prioritized search lasting 10 seconds, and denoising $12 \times 4 = 48$ seconds, while CDGS generation takes $120 \times 4 = 480$ seconds.

5.2.2 How well does extrinsic search scale in inspection planning?

We consider an inspection planning (IP) problem [Fu et al., 2019], in which a drone equipped with a sensor is tasked with observing a set of points of interest (POIs) $\mathcal{P} = \{p_1, \dots, p_k\}$ throughout the environment. While superficially related to multi-goal reaching, IP differs fundamentally in that POIs need not be physically reached but rather observed from feasible vantage points. Consequently, the problem requires jointly optimizing (i) the inspection order of POIs, (ii) the selection of observation viewpoints for each POI, and (iii) the connecting motion between viewpoints.

In our experiments, we consider an inspection of a large-scale bridge by a flying drone (Fig. 1) with $n \in \{4, 8, 16, 64, 128\}$ POIs distributed over the bridge structure using farthest-point sampling over mesh vertices. The drone obeys 3D linear dynamics, where the state consists of position and velocity, and the actions correspond to accelerations along the x , y , and z axes. As a shared base model, we train a diffusion model with the same hyperparameters as used for PointMaze Giant. Full details on data collection, model training and problem formulaion are provided in Appendix B.

To focus on the combinatorial aspects of IP, we apply two simplifying assumptions across all methods. First, we assume perfect tracking at test time and treat the planned trajectory as the executed trajectory. Second, we allow minor collisions, defined as up to one second of contact; episodes are terminated if a collision persists longer than this threshold. For each n POIs we simulate three episodes each starting at different start location, and repeat each episode over three random seeds. We evaluate each method according to the achieved *coverage*—percentage of POIs observed during execution, where an observation is registered once the drone is within some threshold Euclidean distance from the POI.

Our method. We adapt XDiffuser to inspection planning (IP) by decomposing the problem into high-level combinatorial planning and low-level trajectory generation. For each POI, we associate a set of candidate viewpoints by selecting its K nearest neighbors in the state space and augmenting the graph accordingly (Section 4.1). We then apply the mixed-integer linear programming (MILP) Graph-IP solver of Morgan et al. [2026] on the resulting graph to obtain a sequence of vertices forming a valid covering tour over POI viewpoints. This sequence defines a sparse set of waypoints, which we use to guide the diffusion model toward generating a dynamically feasible inspection trajectory. We refer to this instantiation as MILP-XDiffuser.

Baseline. As an intrinsic-search baseline, we adapt CDGS by modifying its search objective to favor inspection coverage by ranking candidate trajectories according to the number of POIs observed,

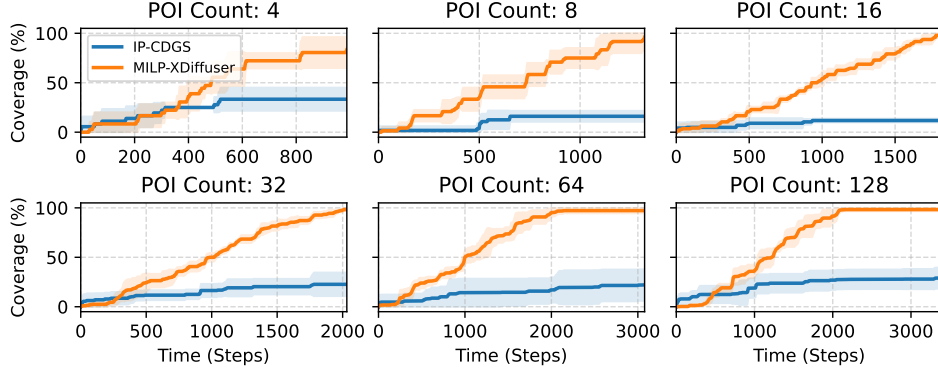


Figure 3: POI coverage over mission time for the inspection-planning task.

Method	4	8	16	32	64	128	Avg.
IP-CDGS	33.3±12.1	12.5±8.6	11.8±2.0	20.1±13.7	22.0±16.4	31.0±10.2	21.8
MILP-XDiffuser	83.3±12.1	95.8±6.1	100.0±0.0	98.6±1.6	97.2±2.9	98.3±1.8	95.5

Table 3: Final POI coverage (%) for the inspection-planning task across different numbers of POIs.

encouraging generation of high-coverage solutions. For a fair comparison, we set the number of CDGS’s trajectory segments to match the number implied by the MILP-XDiffuser plan.

Results. Fig. 3 illustrates how coverage evolves over execution time for each method, with final coverage summarized in Table 3. Although both methods use the same pretrained diffusion model, MILP-XDiffuser consistently achieves near-complete coverage, exceeding 95% on instances with 8 POIs or more, while exhibiting a steady and monotonic increase in coverage throughout execution. In contrast, IP-CDGS exhibits myopic search behavior, with modest early gains but fails to sustain progress, plateauing well below full coverage and never exceeding 50%. These results highlight the benefit of dedicated extrinsic planning for scaling diffusion planners to long-horizon and complex combinatorial problems.

6 Discussion and Conclusion

Limitations. Our graph construction is intentionally simple, relying on sampled states and temporal distance connectivity. While this suffices for the tasks we consider, it may prove to be a hurdle in more complex settings, e.g. with stochastic dynamics. It may also hinder performance when the sampled graph is disconnected, as shown in A.1.1. Moreover, as temporal distance is symmetric, it requires the graph to be undirected, which does not faithfully capture many robotic systems. Incorporating richer graph representations, learned abstractions, or uncertainty-aware connectivity remains an open direction. We look forward to exploring formulations that better align with the diffusion model’s inference, e.g., learn likelihood as graph edge costs, as proposed by Liu et al. [2020].

Conclusion. We studied the problem of long-horizon offline planning from short, suboptimal trajectory data. We identified a key limitation of compositional diffusion and planning-as-inference approaches: when long-horizon behavior is never observed during training, local generative consistency is insufficient to ensure globally coherent plans. To address this, we proposed a simple decomposition: use graph search for global coordination and diffusion for local trajectory generation. Our method XDiffuser constructs a graph over offline states, plans a sparse sequence of waypoints via search, and then guides a pretrained compositional diffusion model to synthesize a continuous, dynamically plausible trajectory. By placing search outside the denoising loop, the approach is both efficient and modular, enabling adaptive horizons and flexible test-time objectives. Empirically, we showed that this combination leads to strong improvements on the OGBench suite, consistently outperforming prior diffusion-based planners and matching or exceeding state-of-the-art value-based methods, particularly in the low-quality-data regime. Moreover, the same framework extends naturally to more complex tasks such as multi-agent path finding and combinatorial routing, simply by changing the high-level graph objective.

References

- Yuze Jiang, Zhouzhou Huang, Bin Yang, and Wenyu Yang. A review of robotic assembly strategies for the full operation procedure: planning, execution and evaluation. *Robotics and Computer-Integrated Manufacturing*, 78:102366, 2022.
- Saeid Nahavandi, Roohallah Alizadehsani, Darius Nahavandi, Shady Mohamed, Navid Mohajer, Mohammad Rokonzaman, and Ibrahim Hossain. A comprehensive review on autonomous navigation. *ACM Computing Surveys*, 57(9):1–67, 2025.
- David Lattanzi and Gregory Miller. Review of robotic infrastructure inspection systems. *Journal of Infrastructure Systems*, 23(3):04017004, 2017.
- Seohong Park, Kevin Frans, Deepinder Mann, Benjamin Eysenbach, Aviral Kumar, and Sergey Levine. Horizon reduction makes RL scalable. In *NeurIPS 2025 Workshop: Second Workshop on Aligning Reinforcement Learning Experimentalists and Theorists*, 2025a.
- T.M. Moerland, J. Broekens, A. Plaat, and C.M. Jonker. *Model-based Reinforcement Learning: A Survey*. Foundations and Trends in Machine Learning Series. Now Publishers, 2023. ISBN 9781638280569. URL <https://books.google.co.il/books?id=hGiVzwEACAAJ>.
- Xu Wang, Sen Wang, Xingxing Liang, Dawei Zhao, Jincai Huang, Xin Xu, Bin Dai, and Qiguang Miao. Deep reinforcement learning: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 35(4):5064–5078, 2024.
- Michael Janner, Yilun Du, Joshua B. Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning*, 2022.
- Matthew Botvinick and Marc Toussaint. Planning as inference. *Trends in cognitive sciences*, 16(10):485–488, 2012.
- Utkarsh Aashu Mishra, Shangjie Xue, Yongxin Chen, and Danfei Xu. Generative skill chaining: Long-horizon skill planning with diffusion models. In *Proceedings of the Conference on Robot Learning (CoRL)*, 2023.
- Yunhao Luo, Utkarsh Aashu Mishra, Yilun Du, and Danfei Xu. Generative trajectory stitching through diffusion composition. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Xiangcheng Zhang, Haowei Lin, Haotian Ye, James Zou, Jianzhu Ma, Yitao Liang, and Yilun Du. Inference-time scaling of diffusion models through classical search, 2025. URL <https://arxiv.org/abs/2505.23614>.
- Utkarsh A Mishra, Dong He, Yilun Chen, and Danfei Xu. Compositional diffusion with guided search for long-horizon planning. *arXiv preprint arXiv:2601.00126*, 2026.
- Jaesik Yoon, Hyeonseo Cho, and Sungjin Ahn. Compositional monte carlo tree diffusion for extendable planning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. Ogbench: Benchmarking offline goal-conditioned RL. In *International Conference on Learning Representations (ICLR)*, 2025b.
- Benjamin J Cohen, Gokul Subramania, Sachin Chitta, and Maxim Likhachev. Planning for manipulation with adaptive motion primitives. In *IEEE International Conference on Robotics and Automation*, pages 5478–5485, 2011.
- Lydia E Kavraki, Petr Svestka, Jean-Claude Latombe, and Mark H Overmars. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Transactions on Robotics and Automation*, 12(4):566–580, 1996.
- Itai Panasoff and Kiril Solovey. Effective sampling for robot motion planning through the lens of lattices. In *Robotics: Science and Systems (RSS)*, 2025.
- Shiyue Wang, Haozheng Xu, Yuhang Zhang, Jingran Lin, Changhong Lu, Xiangfeng Wang, and Wenhao Li. Where paths collide: A comprehensive survey of classic and learning-based multi-agent pathfinding. *ArXiv*, abs/2505.19219, 2025.
- Adir Morgan, Kiril Solovey, and Oren Salzman. Scalable inspection planning via flow-based mixed integer linear programming. In *World Symposium on the Algorithmic Foundations of Robotics (WAFR)*, 2026.
- Nikolay Savinov, Alexey Dosovitskiy, and Vladlen Koltun. Semi-parametric topological memory for navigation. In *International Conference on Learning Representations (ICLR)*, 2018.

- Kuan Liu, Thanard Kurutach, Hsiao-Yu Tung, Pieter Abbeel, and Aviv Tamar. Hallucinative topological memory for zero-shot visual planning. In *International Conference on Learning Representations (ICLR)*, 2020.
- Benjamin Eysenbach, Ruslan Salakhutdinov, and Sergey Levine. Search on the replay buffer: Bridging planning and reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- Linfeng Zhang, Ge Yang, and Bradley C Stadie. World model as a graph: Learning latent landmarks for planning. In *International Conference on Machine Learning (ICML)*, 2021.
- Seungho Baek, Taegeon Park, Jongchan Park, Seungjun Oh, and Yusung Kim. Graph-assisted stitching for offline hierarchical reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2025.
- Johannes Betz, Tobias Betz, Felix Fent, Maximilian Geisslinger, Alexander Heilmeier, Leonhard Hermansdorfer, Thomas Herrmann, Sebastian Huch, Phillip Karle, Markus Lienkamp, et al. TUM autonomous motorsport: An autonomous racing software for the indy autonomous challenge. *Journal of Field Robotics*, 40(4):783–809, 2023.
- Khaled Wahba, Joaquim Ortiz-Haro, Marc Toussaint, and Wolfgang Hönig. Kinodynamic motion planning for a team of multirotors transporting a cable-suspended payload in cluttered environments. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12750–12757. IEEE, 2024.
- Songhao Huang, Yuwei Wu, Yuezhan Tao, and Vijay Kumar. Safe interval motion planning for quadrotors in dynamic environments. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 2780–2786, 2025.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Prafulla Dhariwal and Alexander Nichol. Diffusion models beat gans on image synthesis. *Advances in neural information processing systems*, 34:8780–8794, 2021.
- Stuart J. Russell and Peter Norvig. *Artificial Intelligence: a modern approach*. Pearson, 3 edition, 2009.
- Yaniv Hassidof, Tom Jurgenson, and Kiril Solovey. Train-once plan-anywhere kinodynamic motion planning via diffusion trees. In *Conference on Robot Learning (CoRL)*, 2025.
- Jiayi Liang, Sven Koenig, and Ferdinando Fioretto. Discrete-guided diffusion for scalable and safe multi-robot motion planning. In *AAAI Conference on Artificial Intelligence (AAAI)*, 2026.
- Xiaolin Fang, Caelan Reed Garrett, Clemens Eppner, Tomas Lozano-Perez, Leslie Pack Kaelbling, and Dieter Fox. Dimsam: Diffusion models as samplers for task and motion planning under partial observability. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.
- Sigmund H Høeg, Aksel Vaaler, Chaoqi Liu, Olav Egeland, and Yilun Du. Hybrid diffusion for simultaneous symbolic and continuous planning. *IEEE Robotics and Automation Letters*, 2026.
- Anurag Ajay, Yilun Du, Abhi Gupta, Joshua B. Tenenbaum, Tommi S. Jaakkola, and Pulkit Agrawal. Is conditional generative modeling all you need for decision making? In *International Conference on Learning Representations*, 2023.
- Seohong Park, Tobias Kreiman, and Sergey Levine. Foundation policies with hilbert representations. In *International Conference on Machine Learning*, pages 39737–39761, 2024.
- Jonathan S Yedidia, William T Freeman, and Yair Weiss. Constructing free-energy approximations and generalized belief propagation algorithms. *IEEE Transactions on information theory*, 51(7):2282–2312, 2005.
- Joao Carvalho, An T Le, Mark Baierl, Dorothea Koert, and Jan Peters. Motion planning diffusion: Learning and planning of robot motions with diffusion models. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1916–1923. IEEE, 2023.
- Jiaming Song, Qingsheng Zhang, Hongxu Yin, Morteza Mardani, Ming-Yu Liu, Jan Kautz, Yongxin Chen, and Arash Vahdat. Loss-guided diffusion models for plug-and-play controllable generation. In *International Conference on Machine Learning (ICML)*, July 2023.
- Erwin Coumans and Yunfei Bai. Pybullet, a python module for physics simulation for games, robotics and machine learning, 2016.
- Michael Erdmann and Tomas Lozano-Perez. On multiple moving objects. *Algorithmica*, 2(1):477–521, 1987.

Yorai Shaoul, Itamar Mishani, Shivam Vats, Jiaoyang Li, and Maxim Likhachev. Multi-robot motion planning with diffusion models. In *International Conference on Learning Representations*, 2025.

Mengyu Fu, Alan Kuntz, Oren Salzman, and Ron Alterovitz. Toward asymptotically-optimal inspection planning via efficient near-optimal graph search. In *Proceedings of Robotics: Science and Systems (RSS)*, 2019.

A Appendix

A.1 Ablation study and design choices

Unless otherwise noted, ablations are performed on the AntMaze-Large-Stitch task.

A.1.1 How sensitive is XDiffuser to graph size and connectivity?

We jointly vary the number of sampled vertices n and the graph connectivity parameter k (number of nearest neighbors) to assess sensitivity to graph construction. Table 4 shows a clear threshold behavior: for low connectivity ($k = 10$), performance is poor unless the graph is very dense, indicating failure to capture long-range feasible paths. However, once connectivity increases ($k \geq 20$), success rates rise sharply and quickly saturate, remaining high across a wide range of n . In particular, performance is consistently strong for $k = 30$, with low variance, suggesting reliable recovery of global routes. Beyond moderate coverage, increasing n yields diminishing returns, indicating that the method depends primarily on achieving sufficient connectivity rather than precise graph tuning.

n	k=10	k=20	k=30	k=40
500	0 ± 0	90 ± 9	95 ± 0	90 ± 0
750	0 ± 0	90 ± 13	93 ± 8	88 ± 3
1000	90 ± 5	83 ± 3	95 ± 0	90 ± 13
1500	87 ± 10	82 ± 14	92 ± 3	90 ± 9

Table 4: Success rate across number of graph nodes n and connectivity k .

A.1.2 How sensitive is XDiffuser to the downsampling waypoint interval Δt ?

Δt	8.0	12.0	24.0	48.0
Success rate	93 ± 3	95 ± 0	88 ± 8	92 ± 6

Table 5: Success rate across different waypoint intervals Δt .

We vary the waypoint downsampling interval Δt , maintaining a waypoint every Δt steps along the planned trajectory. To evaluate the impact of this choice, we run experiments on the first 20 episodes of AntMaze Large, repeating each episode across five random seeds. The results indicate a modest trade-off: smaller Δt yields denser supervision but may overconstrain the denoising process, while larger Δt leads to overly sparse waypoints which provide weak guidance. Empirically, $\Delta t = 12$ achieves the best performance, and we adopt this value across all experiments.

A.1.3 Is diffusion necessary after graph search?

We compare the full method against a graph-only execution baseline that feeds the shortest path directly to the inverse dynamics model, without diffusion refinement, demonstrated in Fig 4. Given a sequence of graph waypoints, we linearly interpolate between them to produce a continuous trajectory for execution. Somewhat surprisingly, this simple approach already achieves a $77 \pm 1.4\%$ success rate, indicating that the learned graph provides a strong backbone for high-level planning.

However, its limitations become pronounced in longer-horizon settings. While interpolated waypoints can approximate a smooth trajectory, they often skim obstacles or introduce sharp turns that are difficult to execute robustly. These small tracking errors accumulate over time and are hard to recover from, leading to a sharp drop in performance. In particular, on AntMaze Giant, the graph-only baseline achieves just $6 \pm 0\%$ success. This highlights the necessity of diffusion refinement for producing smooth, dynamically feasible trajectories that remain robust over long horizons.

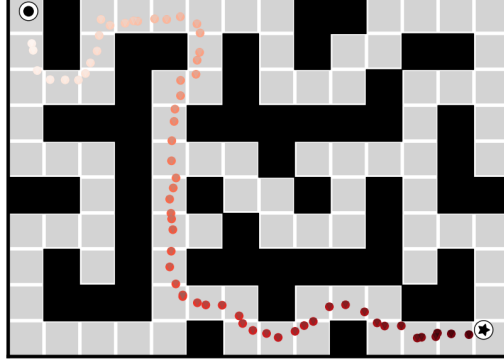


Figure 4: An example XDiffuser graph shortest path, prior to downsampling. Initial state is marked with a black circle, and goal with a star.

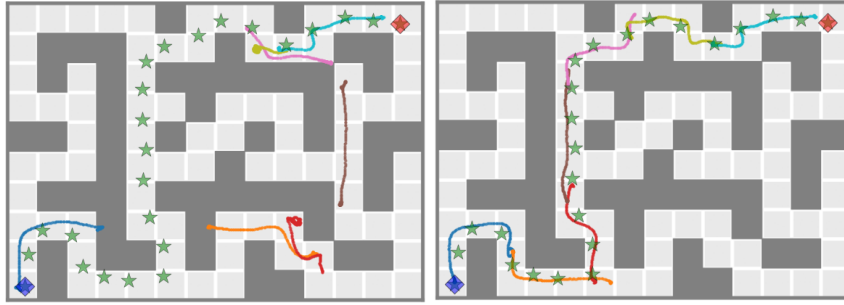


Figure 5: **Guidance window effect.** During graph-guided generation every waypoint attracts states from the generated segments around its nominal time. Left: attracting a single states produces very weak guidance, and as a results segments adhere to their local denoising objective while ignoring the global waypoint structure. Right: by using a triangular guidance window, guidance is distributed along the trajectory creating effective guidance which properly aligns all segments. 5.

B Inspection Planning using XDiffuser

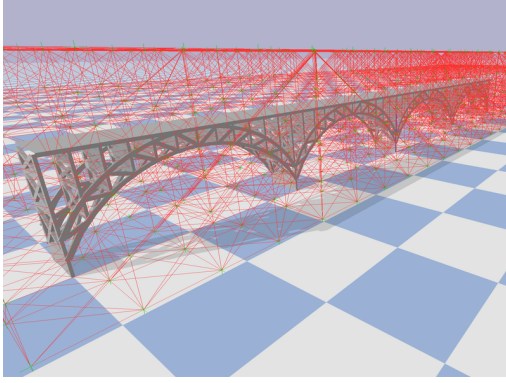
This appendix provides additional details for the inspection-planning experiment described in Sec. 5.2.2. We describe the 3D bridge environment, the drone dynamics, the construction of the offline trajectory dataset, the inspection graph used by MILP-XDiffuser, and the evaluation protocol.

B.1 Offline Dataset Construction

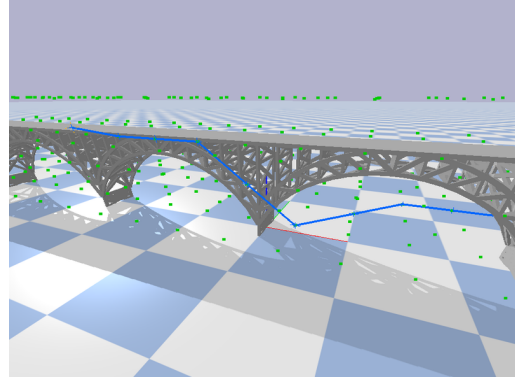
We construct an offline dataset of short dynamically feasible drone trajectories in the bridge environment. The dataset is used only to train the local diffusion trajectory model; it does not contain demonstrations of the inspection-planning task, inspection rewards, or complete tours over POIs. This mirrors the long-horizon setting considered in the main experiments, where the model is trained on local behavior but evaluated on generalization tasks requiring global coordination.

Environment. We use a 3D bridge model represented by a triangular mesh and simulate robot-environment interactions in PyBullet [Coumans and Bai, 2016]. The workspace is obtained from the bridge mesh bounding box, expanded by a fixed margin to allow free-space motion around the structure. All physical sizes are expressed in the normalized coordinate frame of the mesh.

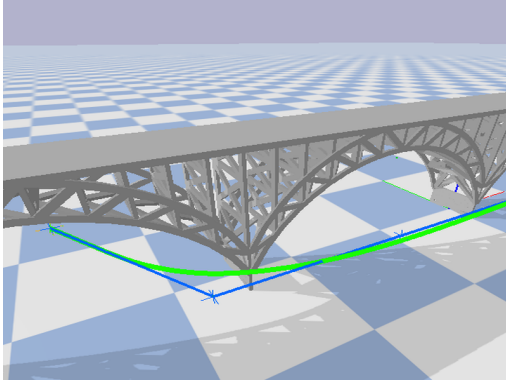
Geometric path generation. To generate candidate motions, we first discretize the collision-free workspace into a 3D grid with resolution $\rho = 0.25$. Collision checking is performed directly against the mesh using a spherical drone body of radius $r = 0.15$. Grid vertices correspond to collision-free positions, and edges connect neighboring vertices under a 18-connected neighborhood when the straight-line segment between them is collision-free, with 5 collision-check samples per edge. For



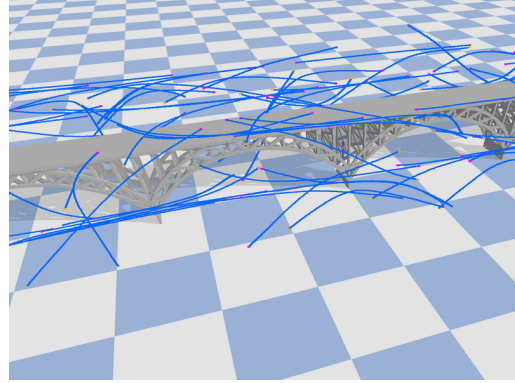
(a) Workspace discretization as a 3D planning graph with collision-free edges.



(b) Geometric path generated on the sampled grid between two randomly sampled positions.



(c) Dynamically feasible trajectory obtained via a realistic high-level drone controller, tracking rolling trajectory waypoints.



(d) Collection of sampled trajectories forming the training dataset.

Figure 6: Dataset generation pipeline.

each dataset trajectory, we generate random collision-free start and goal positions, connect them to the motion planning grid via their six closest nearest grid vertices, and compute a shortest path on this grid using the A* algorithm with Euclidean distance heuristic.

Dynamics and tracking. Each geometric path is converted into a dynamically feasible trajectory using a PID controller applied independently along each axis under a double-integrator drone model. The state is $s_t = (p_t, v_t) \in \mathbb{R}^6$, where $p_t \in \mathbb{R}^3$ is position and $v_t \in \mathbb{R}^3$ is velocity, and the action $a_t \in \mathbb{R}^3$ specifies acceleration. We use the discrete-time dynamics

$$p_{t+1} = p_t + \Delta t v_t, \quad v_{t+1} = v_t + \Delta t a_t,$$

with timestep $\Delta t = 0.05$ s and acceleration clipped to $\|a_t\|_\infty \leq 3$. This dynamics model captures the high-level behavior of a drone while abstracting away platform-specific actuation details, which are assumed to be handled by an inner control loop. The controller tracks the geometric path by selecting accelerations toward a rolling target waypoint. After rollout, each trajectory is clipped to $H_{\text{train}} = 200$ state-action pairs to match the OGBench *Stitch* format [Park et al., 2025b], and is collision-checked again under the executed dynamics; trajectories that collide with the bridge are discarded.

Dataset construction and model training. Repeating the sampling, geometric planning, tracking, and validation procedure yields a dataset of 5,000 state-action trajectories. The dataset contains only short point-to-point motions and does not include POIs, inspection rewards, or complete inspection tours. To avoid directional bias in the generated motions, we sample trajectories within a cubic bounding box rather than using the elongated bounding box of the bridge mesh. We use the state

sequences to train the compositional diffusion model, using the same architecture, denoising objective, and optimization settings as in the main experiments. The same dataset is also used to construct XDiffuser’s extrinsic guidance graph. At test time, inspection-specific structure enters only through the graph-level inspection planner, not through the diffusion training data.

B.2 Inspection-Planning Experiment

We now describe how the trained model is used to solve inspection-planning tasks. Each inspection instance is defined by a set of $n \in \{4, 8, 16, 32, 64, 128\}$ points of interest (POIs) sampled on the bridge surface using farthest-point sampling. A POI is considered observed once the drone comes within distance $r_{\text{obs}} = 1$ of it.

To connect the POIs to XDiffuser’s extrinsic guidance graph, we define an inspection relation between POIs and graph vertices based on spatial proximity. Specifically, for each POI, we associate a set of candidate viewpoint vertices from the graph. This inspection relation defines which graph vertices can observe which POIs.

Given the POI set and the inspection relation, we construct a graph-based inspection-planning problem over the same extrinsic graph used by XDiffuser. We solve this discrete problem using the Graph-IP planner of Morgan et al. [2026], which returns a sequence of graph vertices forming a covering tour: the selected vertices collectively observe all POIs while minimizing travel cost over the graph. Specifically, we use the single-commodity-flow MILP formulation, which is well suited to problem instances at this scale, with a timeout of three minutes.

This high-level tour is then converted into temporal waypoints using cumulative graph edge costs, following the waypoint construction described in Sec. 4. Finally, these waypoints are provided as guidance to the compositional diffusion model, which generates a continuous, dynamically feasible trajectory in the drone state space. We refer to this inspection-planning instantiation as MILP-XDiffuser.

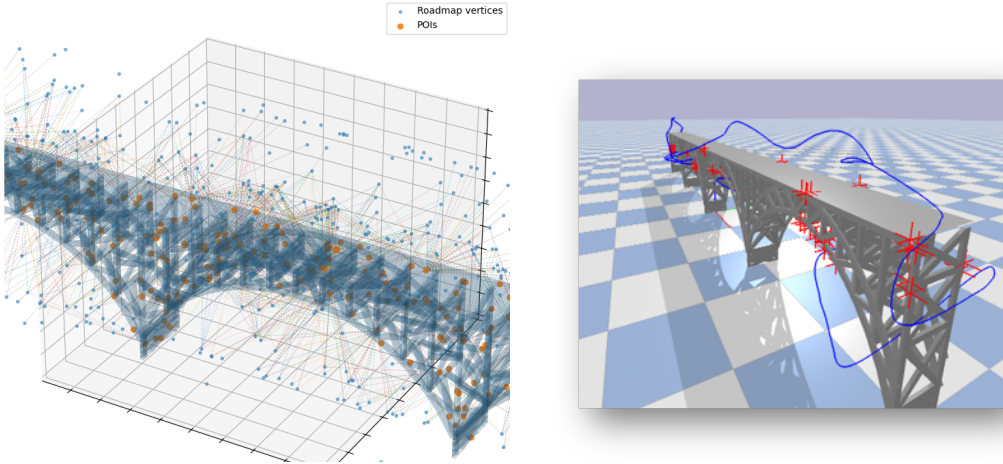


Figure 7: Inspection planning with XDiffuser. (Left) POIs are sampled on the bridge surface and associated with nearby roadmap vertices through the inspection relation. (Right) The graph-level inspection plan is provided as guidance to XDiffuser, which generates a dynamically feasible inspection trajectory.